

Code Profiling and Performance Optimisation

8-Step Practical Exercises

The provided code snippets give you a starting point for your activities. What you need to do is indicated using in-line comments.

Step 1: Establish a baseline

File: **01_establish_baseline.py**

```
import time

def workload(n=200_000):
    total = 0

    for i in range(n):
        total += (i * i) % 97

    return total

if __name__ == "__main__":
    start = time.perf_counter()
    result = workload()
    elapsed = time.perf_counter() - start

    print("result:", result)
    print("elapsed:", elapsed)

    # TASK:
    # Repeat the measurement at least five times.
    # Record the typical elapsed time.
    # Do not change the implementation yet.
```

Task

Record a baseline and explain why measurement must precede optimisation.

Step 2: Build a repeatable benchmark

File: **02_repeatable_benchmark.py**

```
import time

def workload(n=200_000):
    total = 0

    for i in range(n):
        total += (i * i) % 97

    return total

if __name__ == "__main__":
    times = []

    for _ in range(10):
        start = time.perf_counter()
        workload()
        times.append(time.perf_counter() - start)

    print("min:", min(times))
    print("max:", max(times))
    print("avg:", sum(times) / len(times))

    # TASK:
    # Explain why a single timing is unreliable.
    # Consider system load and other sources of measurement noise.
```

Task

Build a repeatable benchmark and explain measurement variability.

Step 3: Profile with cProfile

File: `03_profile_with_cprofile.py`

```
import cProfile
import pstats

def transform(values):
    return [str(v).upper() for v in values]

def process(values):
    output = []

    for v in values:
        # Deliberately repeated work
        output.extend(transform([v]) * 3)

    return output

if __name__ == "__main__":
    values = list(range(20_000))

    profiler = cProfile.Profile()
    profiler.enable()

    process(values)

    profiler.disable()

    stats = pstats.Stats(profiler).sort_stats("cumulative")
    stats.print_stats(10)

    # TASK:
    # Identify the top three functions.
    # Explain ncalls, tottime and cumtime.
```

Task

Use cProfile and interpret ncalls, tottime and cumtime.

Step 4: Inspect callers and callees

File: **04_inspect_callers_callees.py**

```
import cProfile
import pstats

def transform(values):
    return [str(v).upper() for v in values]

def process(values):
    output = []

    for v in values:
        output.extend(transform([v]) * 3)

    return output

if __name__ == "__main__":
    values = list(range(20_000))

    profiler = cProfile.Profile()
    profiler.enable()
    process(values)
    profiler.disable()

    profiler.dump_stats("profile.prof")

    stats = pstats.Stats("profile.prof")
    stats.sort_stats("cumulative")

    stats.print_stats(15)
    stats.print_callers("transform")
    stats.print_callees("process")

    # TASK:
    # Identify which caller is responsible for repeated work.
    # Suggest one change that could reduce calls without
    # changing the result.
```

Task

Use pstats to inspect callers and callees and identify repeated work.

Step 5: Drill into a hot function

File: **05_line_level_profiling.py**

```
def calculate(values):
    result = []

    for value in values:
        cleaned = str(value).strip().lower()
        result.append(cleaned[::-1])

    return result

if __name__ == "__main__":
    values = list(range(100_000))
    result = calculate(values)

    print("items:", len(result))

    # TASK:
    # If line_profiler is installed, profile calculate().
    #
    # Install if required:
    # python -m pip install line_profiler
    #
    # Use the current line_profiler workflow supported by
    # your installed version.
    #
    # Identify the line with the most meaningful contribution
    # to total execution time.
```

Task

Use line-level profiling concepts to locate an expensive line.

Step 6: Investigate memory allocations

File: **06_tracemalloc_memory.py**

```
import tracemalloc

if __name__ == "__main__":
    tracemalloc.start()

    snapshot_before = tracemalloc.take_snapshot()

    # Run the workload being investigated.
    data = [str(i) * 10 for i in range(100_000)]

    snapshot_after = tracemalloc.take_snapshot()

    for stat in snapshot_after.compare_to(
        snapshot_before,
        "lineno",
    )[:10]:
        print(stat)

    # TASK:
    # Identify the largest allocation sites.
    # Decide whether the allocations are necessary.
    # Remember that tracemalloc focuses on Python allocations.
    # It does not account for every native allocation.

    del data
    tracemalloc.stop()
```

Task

Use tracemalloc snapshots to investigate Python memory allocations.

Step 7: Implement and validate an optimisation

File: **07_profile_and_optimise.py**

```
import time

def count_matches(values, targets):
    count = 0

    for value in values:
        if value in targets:
            count += 1

    return count

if __name__ == "__main__":
    values = list(range(200_000))
    targets = list(range(0, 200_000, 10))

    # TASK:
    # 1. Benchmark the baseline.
    # 2. Profile it.
    # 3. Replace the lookup structure if semantics permit.
    # 4. Benchmark again.
    # 5. Assert that both versions return the same count.
    #
    # Explain why the new structure changes expected lookup
    # complexity and quantify the speedup.
```

Task

Profile, optimise and verify a membership-test workload.

Step 8: Produce an evidence-based report

File: **08_evidence_based_report.py**

```
report = {
    "baseline_seconds": None,
    "hotspot": None,
    "hypothesis": None,
    "change": None,
    "optimised_seconds": None,
    "speedup": None,
    "correctness_verified": False,
}

if __name__ == "__main__":
    print(report)

    # TASK:
    # Complete the report using measurements from Steps 1 to 7.
    #
    # Calculate:
    # speedup = baseline_time / optimised_time
    #
    # Include:
    # - benchmark repetitions
    # - machine and Python version
    # - workload size
    # - measurement noise and limitations
    # - the evidence supporting the hotspot
    # - the optimisation and correctness result
```

Task

Produce an evidence-based performance report.